# [Day2] 1. RAG의 개념과 필요성

| | |
|---|---|
| Slug | sds-ax-02-01 |
| Category | |
| Date | 2026년 8월 12일 오후 7:26 |
| module | day2 |
| order | 1 |
| status | 강의실 전용 |
| 강의실(classroom) | sds-ax |

💡 pdf 다운로드

1-1. LLM에 사내 지식이 없는 문제

1-2. RAG 파이프라인

1-3. RAG vs 파인튜닝

## 1-1. LLM에 사내 지식이 없는 문제

💡 환각이 왜 생기는지 확인하고 RAG가 해결하는 지점을 이해해봅니다.

어제 만들었던 사내 문의 응답 체인을 떠올려 보겠습니다. 예상 처리 기한 또는 사내 관련된 내용을 물었더니 모델이 그럴듯한 답변을 지어내거나 모른다는 답을 받으셨을 것입니다. 어느 쪽이든 같은 상황입니다. 근거로 삼을 사내 문서를 준 적이 없습니다.

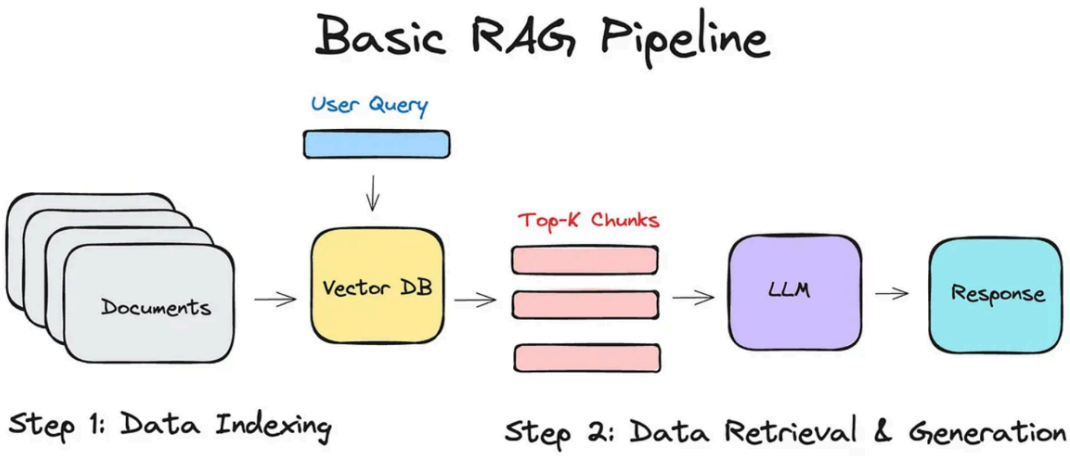
왜 모를까요? 이유는 세 가지입니다.

1. 학습 시점의 한계: 학습이 끝난 시점 이후의 일은 모릅니다.
2. 비공개 자료 접근 불가: 사내 문서와 내부 규정은 인터넷에 없으므로 학습된 적이 없습니다.

3. 환각(hallucination): 모르면 모른다고 하지 않고 그럴듯하게 지어내서 자신 있게 말합니다.

앞의 둘은 모르는 것이라 그나마 낫습니다. 문제는 세 번째입니다. 사내 챗봇이 "출장 식비 한도는 하루 10만원입니다"라고 지어내면 직원은 그 말을 믿고 실제로 10만원을 씁니다. 틀린 답보다 자신 있게 틀린 답이 더 위험합니다.

그러면 어떻게 해야 할까요? 모델을 다시 가르칠 수는 없습니다. 대신 답하기 직전에 관련 문서를 찾아 질문과 함께 넣어 줍니다. 이것이 RAG(Retrieval-Augmented Generation, 검색 증강 생성)입니다.




모델 자체는 건드리지 않습니다. 시험에서 오픈북을 허용하는 것과 같습니다. 학생의 머리를 바꾸는 게 아니라 책을 펴 주는 겁니다. 그래서 오늘은 모델이 아니라 책을 준비하는 법을 배웁니다.

**따라하기**

1. 2일차 실습 폴더를 만들고 1일차의 가상환경과 자격 증명을 이어서 사용합니다. 오늘 만드는 실습 파일은 모두 이 day2_practice 폴더에 저장합니다.

```
# 레포 루트(sds-ax-practice)에서 시작합니다. day1_practice 안에 있다면 cd .. 로 한 단계 올라갑니다
mkdir day2_practice
cd day2_practice
..\.venv\Scripts\activate
# 오늘 쓰는 패키지(langchain-chroma, rank-bm25, kiwipiepy 등)
```

```
는 1일차 requirements.txt에 모두 들어 있습니다
# 빠진 것이 있다면 레포 루트에서: python -m pip install -r req
uirements.txt
# .env는 레포 루트에 있는 것을 그대로 씁니다 (복사 불필요)
```

2. 아래 코드를 day2_practice 폴더에 `hallucination.py`로 저장하고 실행해 RAG 없이 사내 규정을 물어봅니다. `ChatBedrockConverse`는 답변을 문자열이 아니라 블록 리스트로 돌려주는 경우가 있어 `get_text` 헬퍼로 텍스트만 뽑아 씁니다.

```python
# hallucination.py - RAG 없이 사내 규정 질문
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse

load_dotenv()
llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
    temperature=0,
)

def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도 하
므로 텍스트만 모아 반환합니다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block in
content if isinstance(block, dict))
    return content


print(get_text(llm.invoke("삼성SDS 국내 출장 식비 한도는 하루
얼마인가요? 정확한 금액만 답해주세요.")))
print(get_text(llm.invoke("삼성SDS 재택근무는 주 며칠까지 가능
한가요? 규정 조항 번호와 함께 알려주세요.")))
```

`get_text` 안의 `"".join(...)`이 낯설 수 있습니다. 블록 리스트를 돌면서 텍스트만 뽑아 하나로 이어 붙이는 문법입니다. 자바 스트림에서 `map`으로 값을 뽑은 뒤 `Collectors.joining()`으로 합치는 것과 같은 일입니다. 오늘 이후로도 계속 쓰는 헬퍼이니 한 번만 눈에 익혀 두면 됩니다.

3. 답을 관찰합니다. 모른다고 거절하거나 일반론으로 답하거나 구체적인 수치를 지어냅니다. 어느 쪽이든 사내 문서를 근거로 한 답이 아닙니다.

**막힐 때**

- 모델이 계속 모른다고만 답합니다.

  최근 모델은 모르는 질문을 잘 거절해서 환각이 바로 재현되지 않을 수 있습니다. 구체적인 조항 번호나 정확한 금액을 요구하는 식으로 질문을 바꾸면 지어내는 경우를 재현할 수도 있습니다. 어떤 답이 와도 문서 근거가 없다는 점은 같습니다.

- activate에서 이 시스템에서 스크립트를 실행할 수 없다는 오류(PSSecurityException)가 납니다.

  PowerShell 창을 새로 열 때마다 다시 걸립니다. `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`를 한 번 실행한 뒤 다시 activate 합니다.

- 자격 증명 오류가 납니다.

  레포 루트에 `.env`가 있는지, 그리고 스크립트를 `day2_practice` 안에서 실행하고 있는지 확인합니다. `load_dotenv()`는 실행한 파일 위치에서 위로 올라가며 찾습니다.

**확인**: 모델의 답이 모른다는 거절이든 일반론이든 사내 규정의 실제 수치를 근거 있게 답하지 못하는 것을 보면 성공입니다.

> 💡 LLM은 사내 문서를 본 적이 없어 검증 장치 없이는 사내 질문의 답을 신뢰할 수 없다는 걸 확인했습니다.

## 1-2. RAG 파이프라인

> 💡 RAG를 구성하는 6단계 파이프라인이 각각 무엇을 하는지 이해해봅니다.

"답하기 직전에 문서를 찾아 넣는다"를 실제로 만들려면 무엇이 필요할까요? 찾으려면 찾을 곳이 먼저 있어야 합니다. 그래서 RAG는 크게 두 단계로 나뉩니다. 문서를 미리 준비해 두는 인덱

싱 단계와, 질문이 들어왔을 때 찾아서 답하는 질의 단계입니다.




각 단계가 하는 일과 담당하는 LangChain 도구는 다음과 같습니다.

| 순서 | 단계 | 하는 일 | LangChain 도구 |
|---|---|---|---|
| 1 | Load | 파일에서 텍스트 꺼내기 | DocumentLoader (TextLoader 등) |
| 2 | Split | 긴 문서를 작은 조각(청크)으로 나누기 | TextSplitter |
| 3 | Embed | 텍스트를 의미 벡터(숫자 배열)로 바꾸기 | Embeddings |
| 4 | Store | 청크와 벡터를 DB에 저장 | VectorStore (Chroma 등) |
| 5 | Retrieve | 질문과 가장 비슷한 청크 N개 찾기 | Retriever |
| 6 | Generate | 찾은 청크를 프롬프트에 넣어 답 생성 | LCEL 체인 |

여기서 두 단계를 나눈 이유가 보입니다. 1단계부터 4단계까지는 문서가 바뀔 때만 다시 돌리면 됩니다. 5단계와 6단계는 질문이 들어올 때마다 돌아갑니다. 비용이 드는 임베딩을 매번 하지 않아도 되는 것은 이 분리 덕분입니다.

오늘 하루 종일 나올 용어 다섯 개를 먼저 잡고 가겠습니다.

- Document: 문서 한 개입니다. 본문 문자열(`page_content`)과 부가 정보(`metadata`)로 구성됩니다.
- Chunk(청크): 문서를 잘게 쪼갠 조각입니다. 검색 단위이자 LLM에 넣는 단위입니다.
- Embedding(임베딩): 텍스트를 고정 길이 벡터로 바꾼 것입니다.
- Vector Store(벡터 저장소): 청크와 임베딩을 저장하고 검색하는 DB입니다.
- Retriever(검색기): 질문을 받으면 관련 청크 N개를 돌려주는 부품입니다.

이 여섯 단계를 오늘 순서대로 만듭니다. 2번째 세션에서 Load와 Split, 3번째 세션에서 Embed와 Store, 4번째 세션에서 Retrieve와 Generate입니다. 4번째 세션이 끝나면 사내 규정을 근거 있게 답하는 체인이 손에 남습니다.

💡 RAG가 인덱싱과 질의 두 단계로 나뉜 6단계 파이프라인이라는 것을 확인했습니다.

## 1-3. RAG vs 파인튜닝

💡 사내 지식을 다룰 때 RAG와 파인튜닝을 언제 각각 쓰는지 판단 기준을 이해해봅니다.

여기서 "그냥 모델에 사내 문서를 학습시키면 되는 거 아닌가요?" 하는 분이 꼭 계십니다. 됩니다. 그게 파인튜닝(fine-tuning, 모델을 사내 데이터로 추가 학습)입니다. 문제는 그 다음입니다. 두 방식을 나란히 놓고 보겠습니다.

| 비교 항목 | RAG | 파인튜닝 |
|---|---|---|
| 지식 갱신 | 문서만 다시 인덱싱하면 즉시 반영 | 다시 학습해야 함 (시간과 비용이 큼) |
| 출처 표시 | 가능 (어느 문서에서 왔는지 추적) | 불가능 (모델 안에 녹아 있음) |
| 초기 비용 | 낮음 (임베딩 비용 정도) | 높음 (GPU, 데이터 정제, 학습) |
| 잘하는 것 | 사실 기반 질의응답 | 말투, 형식, 도메인 어휘 습득 |
| 환각 억제 | 문서에 근거하게 강제 가능 | 학습해도 환각은 남음 |
| 접근 제어 | 검색 단계에서 권한별 필터 가능 | 모델이 통째로 알고 있어 어려움 |

표를 한 줄로 줄이면 이렇습니다. 무엇을 아는가를 바꾸고 싶으면 RAG이고, 어떻게 말하는가를 바꾸고 싶으면 파인튜닝입니다.

- 지식은 자주 바뀝니다. 그래서 문서로 관리하는 편이 낫고, 그게 RAG입니다.
- 말투와 문체는 잘 안 바뀝니다. 법률 문체나 의료 용어처럼 스타일을 몸에 배게 하고 싶을 때는 파인튜닝이 좋은 선택일 수 있습니다.
- 둘 중 하나를 고르는 문제는 아닙니다. 파인튜닝한 모델 위에 RAG를 얹는 조합도 흔합니다.

그러면 기업에서는 무엇부터 볼까요? 대부분 RAG부터 검토합니다. 사내 문서가 가진 세 가지 성격이 전부 RAG 쪽을 가리키기 때문입니다.

- **갱신 비용**: 사내 규정은 자주 바뀝니다.
- **출처 표시**: 답변에 근거를 요구받습니다.
- **접근 제어**: 부서별로 볼 수 있는 문서가 다릅니다.

참고할 만한 영상도 함께 둡니다. 영상의 요지는 다음과 같습니다.

- **기본 모델 + 프롬프트/컨텍스트 엔지니어링**으로 시작
- 최신/자체 데이터가 필요하다면 **RAG** 도입
- 절차적 수행 능력이 필요하다면 **에이전트 스킬** 부여
- 위 단계로도 해결되지 않는 **지연 시간/비용 등의 한계가 존재할 때만 파인튜닝** 진행

https://www.youtube.com/watch?v=-W2JdSl1v48

💡 상황에 따라 RAG 또는 파인튜닝 모델로 가는 판단 기준을 익혔습니다.